

MODE COLLAPSE IN RLVR & MODEBENCH

Anonymous authors

Paper under double-blind review

ABSTRACT

A model can answer more reliably while losing alternative correct solutions. With binary rewards, reinforcement learning with verifiable rewards (RLVR) gives a repeated success and a different success the same feedback. We introduce MODEBENCH, five executable domains that identify distinct verified outputs for the same prompt, and ReplayMaxRL, which combines learning from fresh samples with memory of verified successes. The same execution keys serve both measurement and replay: each retained key receives equal weight, even when it becomes rare in fresh rollouts. Across three model scales, adding replay to Dr.GRPO improves held-out correctness and sampled verified support. Replay also complements MaxRL across all three scales, with five paired seeds in every domain. Harder-level comparisons cover four complete domains, with PantryPlan remaining partial. Hosted-model evaluation also finds accurate answers concentrated on few verified modes. An idealized analysis explains why binary rewards can amplify common modes and how replay can counter that pressure. Together, these results distinguish reliable answering from the availability of verified alternatives.

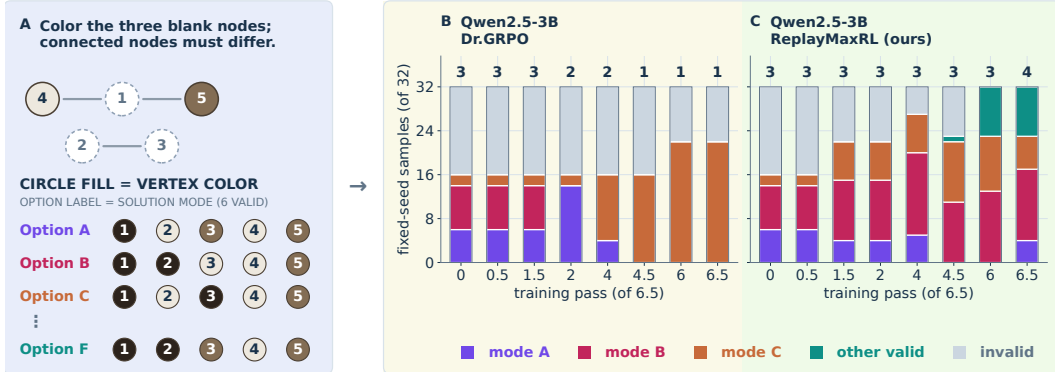


Figure 1: **Correctness can rise while sampled alternatives narrow.** One graph admits six valid colorings. Bars pool four eight-sample draws from matched Qwen2.5-3B runs. At pass 6.5, Dr.GRPO produces 22/32 correct outputs from one mode; ReplayMaxRL produces 32/32 across four. Circle fills mark vertex colors; dashed rings mark vertices left uncolored in the prompt. This illustrates the distinction; Figure 5 separates the effects of the fresh objective and replay.

1 INTRODUCTION

A mathematical agent often works by proposing a computation or proof route, checking it, and trying again after feedback (Brown et al., 2024; Snell et al., 2024). A different route may offer a way forward when the first fails or the problem’s constraints change. A verifier can assess these alternatives only if the model still proposes them. This motivates a question about training: as a model becomes more successful, what happens to the range of successful behaviors it can produce?

Reinforcement learning with verifiable rewards (RLVR) uses executable correctness feedback to improve a model (Guo et al., 2025). A binary reward does not distinguish a familiar correct solution from another correct solution to the same problem. The model can therefore become more accurate while its verified outputs concentrate on fewer alternatives. Figure 1 shows the

distinction on one prompt. Across our benchmark, verifier-only training reduces the number of sampled correct modes beyond the first at all three model scales, although correctness and raw mode counts need not decline in every comparison (Appendix F.2).

The missing measurement is the identity of a successful output. Different strings may encode the same computation, while different computations may reach the same answer. Neither token entropy nor answer accuracy alone tells us which verified alternatives remain available. MODEBENCH addresses this by having each validator return a canonical execution key alongside correctness. These keys define task-specific output modes, allowing evaluation to measure whether sampling succeeds and how many distinct verified outputs it finds.

The same identity also gives training a useful memory. ReplayMaxRL stores one verified exemplar per admitted key and assigns equal replay weight to each retained key. MaxRL learns from fresh samples; replay continues to train on stored successes when they become rare in those samples. Ordinary success replay can reuse correct responses without canonical keys. Our use of keys makes distinct verified outcomes measurable and prevents repeated discoveries of a common outcome from dominating the memory’s weights.

We make three connected contributions: an executable benchmark for measuring verified alternatives, a method that uses those identities for balanced replay, and matched experiments that separate the fresh learning objective from memory. The evidence supports a practical claim: replay can improve both held-out correctness and sampled verified breadth, including beyond a stronger fresh objective. Whether preserving these alternatives improves later search, revision, or adaptation remains a question for further work.

2 MODEBENCH: MEASURING SUCCESSFUL ALTERNATIVES

For a prompt x with specification s_x , a validator maps an output y to failure $\perp$ or a canonical key $V(y, s_x) \in \mathcal{C}_x$. Two accepted outputs count as the same mode exactly when their keys agree. Training sees only keys generated by the current policy; neither the full valid-key set $\mathcal{C}_x$ nor its size is supplied to the learner or replay bank.

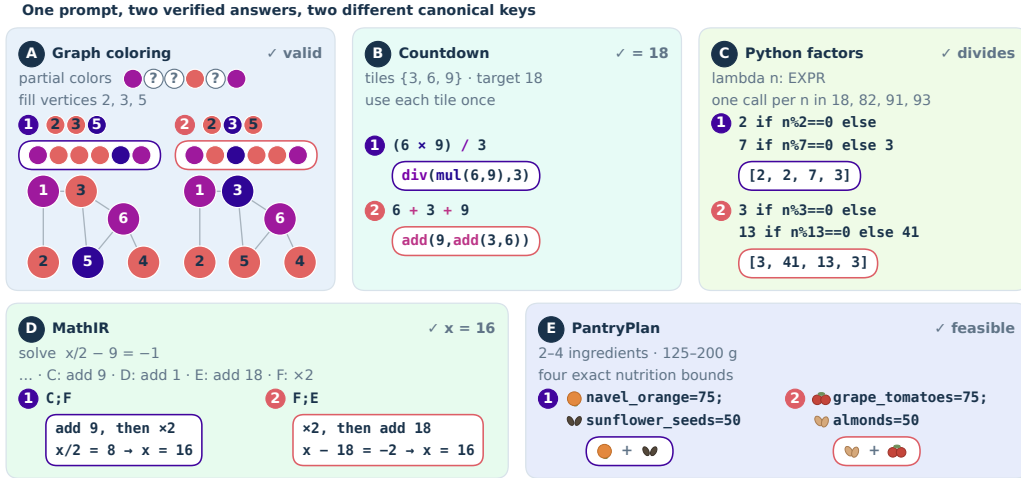


Figure 2: **One prompt can have different verified outputs.** Keys identify a color vector in Graph, a normalized computation tree in Countdown, an executed return vector in Python, an algebraic state trajectory in MathIR, or an ingredient support in PantryPlan. Cosmetic aliases merge; different accepted computations or configurations remain distinct under each domain’s execution rule.

Figure 2 shows the five domains. Countdown and MathIR capture different routes to an answer, Python captures different tested behaviors, and Graph and PantryPlan capture different valid configurations. This distinction matters when interpreting breadth: an additional valid configuration and an additional computation route are both verified modes, but they are not the same kind of alternative. The keys

measure executed outputs, not latent reasoning strategies or the usefulness of a solution in a later task. Appendix B gives the exact key and alias rules, with verbatim prompts in Appendix C.

2.1 SUCCESS AND SAMPLED BREADTH

For K independent outputs, let D_K count their distinct valid keys. Our two primary metrics ask complementary questions:

$$\text{pass}@K = \Pr[D_K > 0], \quad \text{distinct}@K = \mathbb{E}[D_K].$$

The first measures the chance of finding any correct output; the second measures how many different correct outputs the samples contain. We use $K = 8$ throughout the training comparisons. The metrics have different units: probability and expected verified-key count. Their numerical magnitudes should not be compared directly, and a larger mode count can partly reflect more successful samples rather than a more balanced correct-output distribution.

To make that coupling visible, we also report *extra modes*, $\text{distinct}@8 - \text{pass}@8$: the expected number of verified keys beyond the first correct output in a sampled set. This decomposition helps interpret a breadth gain but does not hold correctness fixed. None of these finite-sample measurements identifies the total number of positive-probability modes or proves that an unobserved mode has become impossible. At fixed correctness, a more concentrated allocation reduces expected breadth in the majorization sense (Lemma A.1); increasing correctness alone can also increase it. Appendix A gives the derivation.

For the conditional distribution q over correct keys, we also measure $C(q) = \sum_c q_c^2$: the probability that two independent correct outputs share a key. This connects hosted observations, saved training outputs, and the idealized collapse flow (Appendix F.1).

2.2 HARDER TASKS WITH THE SAME NOTION OF A MODE

The training experiments use two disjoint levels with the same validators and execution keys. Level 2 adds structural difficulty: Graph hides more vertices and Countdown requires more operands. Valid-mode-count histograms match the designated construction references, and frozen-model development evaluation checks solvability. The terminal test populations differ between levels, so their contrast does not isolate difficulty at fixed test-set support (Appendix B.3).

3 REPLAYMAXRL: LEARN FROM NEW SAMPLES AND REMEMBER SUCCESSES

ReplayMaxRL uses the two outputs of the validator for different purposes: correctness rewards train on fresh samples, and canonical keys organize memory. Both paths use policy-generated responses. The method needs neither gold solutions nor a precomputed catalogue of valid modes.

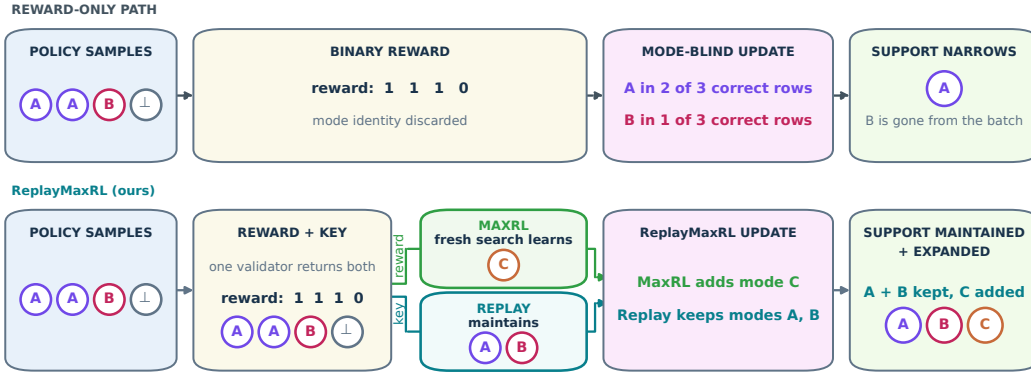


Figure 3: **Fresh learning and memory use the same verified samples.** MaxRL learns from their binary rewards. Replay stores exemplars by canonical key and revisits retained keys with equal weight. In the schematic, fresh samples add C while replay revisits A and B. A stored exemplar can continue to receive a training signal after it becomes rare in fresh samples.

3.1 LEARNING FROM FRESH ROLLOUTS

MaxRL combines success objectives across sampling budgets (Tajwar et al., 2026). With binary rewards, its centered group estimator weights a correct response more strongly when successes in the group are scarce. This provides a useful test of replay: memory should be evaluated alongside an objective designed for learning from sampled successes, as well as alongside Dr.GRPO (Liu et al., 2025).

Both fresh objectives still depend on what the policy currently samples. An absent mode contributes no direct score term, and an all-correct or all-incorrect group has zero centered task advantage. MaxRL changes the fresh learning signal without creating a persistent record of which correct behaviors have occurred. Appendix D.1 specifies the exact advantages and their finite-group interpretation.

3.2 REPLAYING DISTINCT VERIFIED SUCCESSES

Each training prompt owns an initially empty, bounded bank $\mathcal{B}_x$. When a new key is admitted, the bank retains one verified exemplar e_c . Repeated discoveries of that key do not add further entries. A deterministic schedule revisits nonempty prompt banks throughout training. For an exemplar of token length $|e_c|$, the replay loss is

$$L_{\text{replay}}(x) = -\frac{1}{|\mathcal{B}_x|} \sum_{c \in \mathcal{B}_x} \frac{\log \pi_{\theta}(e_c | x)}{|e_c|}. \quad (1)$$

Each retained key receives the same share of replay, independent of how often it appeared in fresh rollouts. A mode discovered once therefore continues to receive a signal alongside a mode discovered repeatedly. Token normalization sets the loss scale for each exemplar; it does not make exemplar likelihood equal to the total probability of generating its canonical key.

We add this loss to the fresh MaxRL update. Replacing MaxRL with Dr.GRPO yields ReplayDr.GRPO. Replay exemplars stay outside the fresh reward group, so the fresh objective and replay can be varied separately. The controls execute the same bank and scoring code with zero replay gradient; their learned banks and workloads can differ as policies diverge. Banks contain only responses to training prompts. Held-out evaluation therefore tests what training changes in the policy, rather than retrieving answers from the bank. Appendix D gives admission, scheduling, masking, and loss scaling.

3.3 WHY MEMORY ADDRESSES A DIFFERENT PRESSURE

Common correct modes appear in more on-policy updates than rare correct modes, although both receive the same binary reward. In an isolated softmax model with one independently

trained logit per canonical outcome and common length normalization, Euclidean mean flows for GRPO, Dr.GRPO, and binary MaxRL concentrate on the initially most probable correct mode when it is unique (Theorem K.3). A fixed bank with constant positive replay dose supplies a qualitative probability floor for its banked modes (Theorem K.6).

Theory studies idealized mean flows. The result explains a pressure that replay can counter; it does not certify the implemented neural optimizer. A finite bank protects only admitted modes in the ideal analysis, and its probability floor can be too small to yield observable samples. In this same fixed-bank flow, unbanked correct modes can vanish. Our experiments therefore measure held-out sampled breadth and inspect exemplar-score trajectories separately. Appendix J identifies the basis and scope of each central claim.

4 EXPERIMENTAL DESIGN

We compare Qwen2.5-0.5B, Falcon3-1B, and Qwen2.5-3B on all five Level-1 domains. Each run trains for eight passes with five registered seeds per comparison. Evaluation averages four eight-sample draws on 128 held-out prompts, with matched requests across methods. Complete paired blocks receive nominal 95% intervals; incomplete intersections retain their exact seed counts and are descriptive. Missing or ambiguous endpoints are not imputed. The shared recipe and source-integrity rules are in Appendices B.4 and L.1.

The comparisons answer three questions. First, does adding canonical replay to Dr.GRPO improve correctness and verified support across scales? Supporting controls test reward normalization, on-policy redistribution, ordinary success memory, and semantic exploration. UCPO and sparse RLEP-Dr are compared at 0.5B and 1B only; fixed Semantic-MaxEnt is defined in Appendix D.5. Second, does replay still help when the fresh objective is MaxRL? The central factorial crosses Dr.GRPO/MaxRL with replay off/on at all three scales, yielding Dr.GRPO, ReplayDr.GRPO, MaxRL, and ReplayMaxRL. Third, does the same pattern hold on the harder Level 2? This factorial uses Qwen2.5-0.5B, with four complete domain comparisons and partial PantryPlan results. Appendix Table 4 gives the exact coverage.

5 RESULTS

Mode collapse is measurable during training. A retrospective analysis of saved outputs finds increased correct-key collision in Graph and PantryPlan after Dr.GRPO and GRPO at all three scales: twelve positive before/after contrasts. ReplayDr.GRPO reduces conditional concentration relative to Dr.GRPO in the same six domain-scale comparisons; both disjoint-stream checks agree in direction. These primary nominal intervals exclude zero. The conclusions concern jointly observable correct outputs, with explicit eligibility and overlapping-seed treatment; Python usually lacks enough initial correct samples, and some other comparisons are sensitive to stream selection. The same eligible prompts can also have lower per-sample correctness with replay. Thus conditional concentration and the full-test success/breadth outcomes below remain distinct empirical claims (Appendix F.1).

5.1 EXPERIMENT 1: REPLAY IMPROVES SUCCESS AND SAMPLED SUPPORT

Replay improves success and sampled support across scales. Figure 4A shows positive block-mean changes in both primary endpoints for every model and domain. Raw `distinct@8` increases in all 74 admissible seed pairs. This consistency matters: the result spans valid configurations, computation routes, and tested program behaviors, rather than depending on a single task or favorable seed. The excluded Falcon Countdown run and its four-seed comparison remain explicit in the figure and Appendix L.1.

Countdown provides a concrete interpretation: replay more than triples the number of sampled verified computation routes at Qwen2.5-0.5B while also raising correctness. Python shows why the two metrics must be read together: its correctness improves substantially, but the gain in breadth beyond the first correct output remains uncertain. More successful samples alone do not demonstrate a more balanced distribution over correct outputs. The appendix reports the exact means and uncertainty for these examples (Appendix E.2).

A ReplayDr.GRPO (ours) – Dr.GRPO across three model scales

	$\Delta \text{pass}@8$							$\Delta \text{distinct}@8$					
Qwen 0.5B	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	Qwen 0.5B	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
Falcon 1B	+0.15	+0.31 [†]	+0.98	+0.64	+0.12	+0.44 [†]	Falcon 1B	+0.68	+0.42 [†]	+1.11	+0.66	+0.94	+0.77 [†]
Qwen 3B	+0.15	+0.14	+0.59	+0.41	+0.11	+0.28	Qwen 3B	+0.58	+0.33	+0.78	+0.43	+0.67	+0.56
	Graph	Countdown	Python	MathIR	Pantry	Average		Graph	Countdown	Python	MathIR	Pantry	Average

B Initial reference + alternatives – terminal Dr.GRPO at Qwen2.5-0.5B

	$\Delta \text{pass}@8$							$\Delta \text{distinct}@8$					
Before training	+0.13	-0.38	-0.17	-0.29	+0.38	-0.07	Before training	+0.24	-0.38	-0.17	-0.29	+1.90	+0.26
ReplayDr.GRPO (ours)	+0.65	+0.19	+0.36	+0.28	+0.21	+0.34	ReplayDr.GRPO (ours)	+2.11	+1.15	+0.40	+0.32	+1.02	+1.00
MaxRL (no replay)	+0.21	+0.10	+0.00	-0.07	+0.01	+0.05	MaxRL (no replay)	+0.35	+0.13	+0.00	-0.07	+0.01	+0.09
UCPO	-0.00	+0.13	+0.00	+0.03	+0.05	+0.04	UCPO	-0.00	+0.18	+0.00	+0.03	+0.05	+0.05
RLEP-Dr	+0.10	+0.10	+0.00	+0.13	+0.00	+0.07	RLEP-Dr	+0.15	+0.13	+0.00	+0.12	+0.00	+0.08
Semantic-MaxEnt	+0.00	+0.14	+0.00	-0.12	+0.20	+0.04	Semantic-MaxEnt	+0.00	+0.18	+0.00	-0.12	+1.12	+0.24
GRPO	+0.04	+0.16	+0.00	-0.05	+0.02	+0.03	GRPO	+0.05	+0.17	+0.00	-0.05	+0.02	+0.04
	Graph	Countdown	Python	MathIR	Pantry	Average		Graph	Countdown	Python	MathIR	Pantry	Average

Figure 4: **Canonical replay improves correctness and verified breadth.** (A) ReplayDr.GRPO minus Dr.GRPO at all three scales. (B) Qwen2.5-0.5B alternatives minus Dr.GRPO. Columns measure probability points and expected verified-key counts, respectively. The green Average weights domains equally within seed. A dagger marks four paired seeds in Falcon Countdown and its Average; other cells use five. Only complete blocks receive nominal 95% intervals. Purple denotes ReplayDr.GRPO; bold entries mark column maxima in Panel B.

The controls also distinguish the value of balancing keys from simply reusing successes. At Qwen2.5-0.5B, ReplayDr.GRPO has the largest mean correctness gain among the direct alternatives in every domain (Figure 4B). Replacing equal key weights with weights proportional to fresh discovery frequency reduces extra modes on average, while the aggregate correctness contrast remains inconclusive. The advantage is concentrated in Graph, Countdown, and PantryPlan; it is not a uniform benefit across tasks. This ablation supports key balancing as a contributor to breadth (Appendix E.6).

5.2 EXPERIMENT 2: REPLAY COMPLEMENTS A STRONGER FRESH OBJECTIVE

Replay adds value beyond MaxRL at all three scales. The two replay comparisons in Figure 5 separate memory from the choice of fresh objective. At 0.5B, 1B, and 3B, replay improves the domain-averaged correctness and raw mode count under both Dr.GRPO and MaxRL. Thus the replay benefit persists when fresh learning already uses MaxRL’s sampling-aware objective. The comparison identifies the effect of adding the replay update; it does not establish which individual stored modes account for the held-out gains.

The effect is not identical across domains or scales. ReplayMaxRL improves both endpoints in every 0.5B domain. At 1B, mean correctness improves in every domain, but Graph’s mean raw-support effect is negative and inconclusive. The 3B MaxRL comparison is now complete: all 50 endpoints cover five domains, both methods, and seeds 70–74. Averaging domains equally within seed, correctness rises from .547 to .701 and verified modes from .744 to 1.107. All five domains have positive mean effects on both endpoints; Graph and Python correctness intervals still include zero. Appendix E.2 gives the aggregate estimates; Appendix E.4 shows both metrics throughout training.

5.3 EXPERIMENT 3: REPLAY ON HARDER MATCHED TASKS

Replay improves both average endpoints at both difficulty levels. Figure 6 compares the four domains with complete factorials at Qwen2.5-0.5B: Graph, Countdown, Python, and MathIR (80/80 endpoints). Their aggregate moves toward greater correctness and sampled support with replay under either fresh objective, at both levels. The levels share mode definitions but use separate test populations; the within-level replay comparisons control prompts and fresh objectives. PantryPlan remains outside this aggregate because its Level-2 factorial is incomplete (10/20 endpoints).

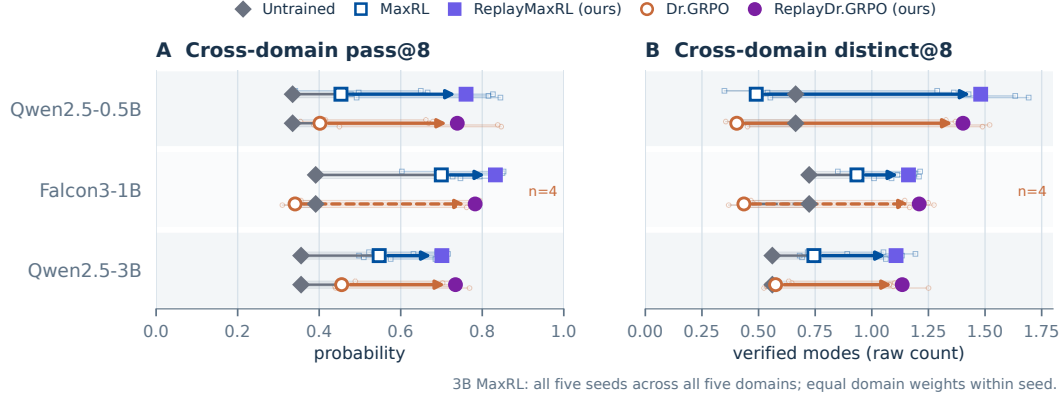


Figure 5: **Replay complements both fresh objectives at multiple scales.** Each row shows Dr.GRPO/ReplayDr.GRPO and MaxRL/ReplayMaxRL on both endpoints. Diamonds are initial models; open/filled markers are training without/with replay. Complete tracks average domains within seed; faint paths show seeds. All MaxRL tracks use five paired seeds in all five domains, including 3B. Falcon Dr.GRPO uses four common seeds; the other Dr.GRPO tracks use five. Appendix Figure 9 reports each domain’s correctness and verified-mode results at every scale.

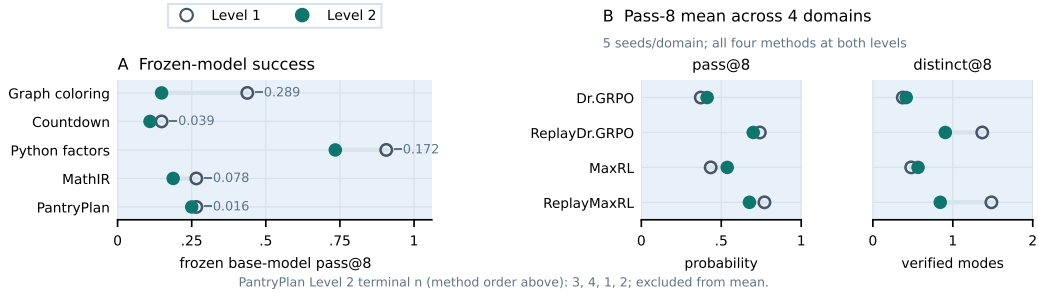


Figure 6: **Replay across difficulty levels at Qwen2.5-0.5B.** Open/filled markers denote Level 1/2. (A) Frozen-model development admission in all five domains. (B) Terminal means over Graph, Countdown, Python, and MathIR, weighted equally within each of five paired seeds. Both levels and all four methods share this completed cohort. Partial PantryPlan arm counts are reported in Appendix E.3, with the construction and test populations distinguished in Appendix B.3.

Graph supplies the clearest domain-level evidence: both replay variants improve correctness and raw support substantially on the harder problem. The other completed domains’ correctness and raw-support intervals include zero. We therefore interpret the aggregate as evidence that replay can remain useful on these harder tasks; the cross-level comparison does not isolate a causal difficulty effect or establish universal improvement. Appendix E.3 reports the Graph estimates, and Appendix E.3 gives all fresh-objective and replay-interaction contrasts.

5.4 HOSTED MODELS: CORRECTNESS DOES NOT DETERMINE BREADTH

The distinction measured by MODEBENCH also appears beyond the training comparisons. We evaluate seven hosted deployments on all five domains and three levels, without additional training. Figure 7 places per-response correctness beside the number of verified modes found in eight samples. Graph and MathIR show why both matter: highly accurate outputs often revisit only one or a few verified modes. PantryPlan yields broader sampled outputs, with substantial variation across deployments and levels.

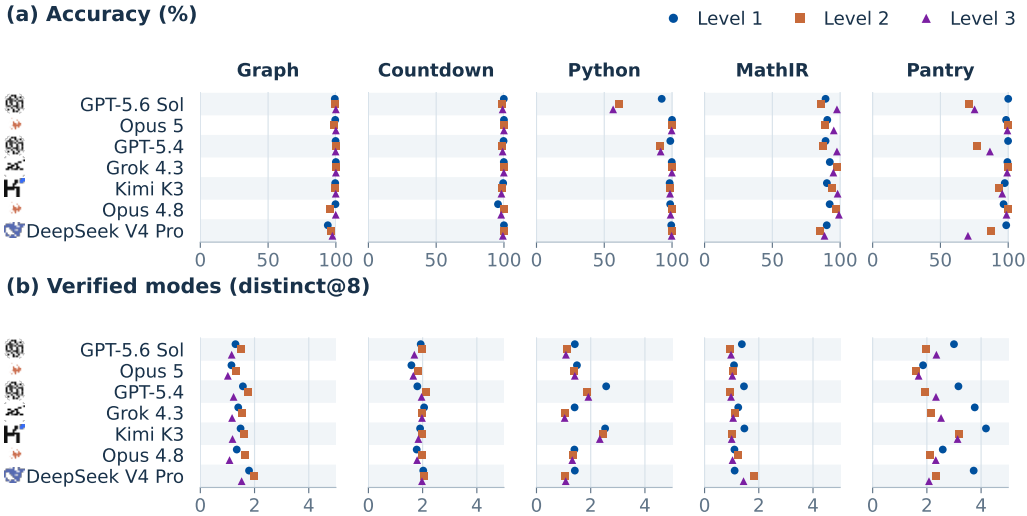


Figure 7: **Accurate hosted outputs can still cover few verified modes.** Columns show domains; markers distinguish Levels 1–3. Each point uses all eight responses to each of 128 prompts. Top: per-response accuracy. Bottom: `distinct@8`. A fixed formatting normalizer applies to every deployment; Opus 5 Python uses revised task wording without a system message. Appendix H gives original-protocol results, prompt comparisons and uncertainty.

These exploratory comparisons describe the evaluated deployment and prompt configurations; they do not identify a training cause or a model-size effect. Higher benchmark levels need not lower correctness for every evaluated deployment.

A matched subset tests four temperatures with GPT-5.6 Sol (Figure 8). Non-default temperatures require reasoning none on this deployment. In that profile, increasing T from .5 to 1.5 raises normalized empirical `pass@8` from 66.7% to 72.5% and mean `distinct@8` from .775 to 1.050. At $T = 2$, these are 70.8% and 1.042; $T = 1.5$ is the best observed aggregate point, not an established optimum. The historical medium-reasoning reference reaches 97.5% and 1.550 and is shown separately.

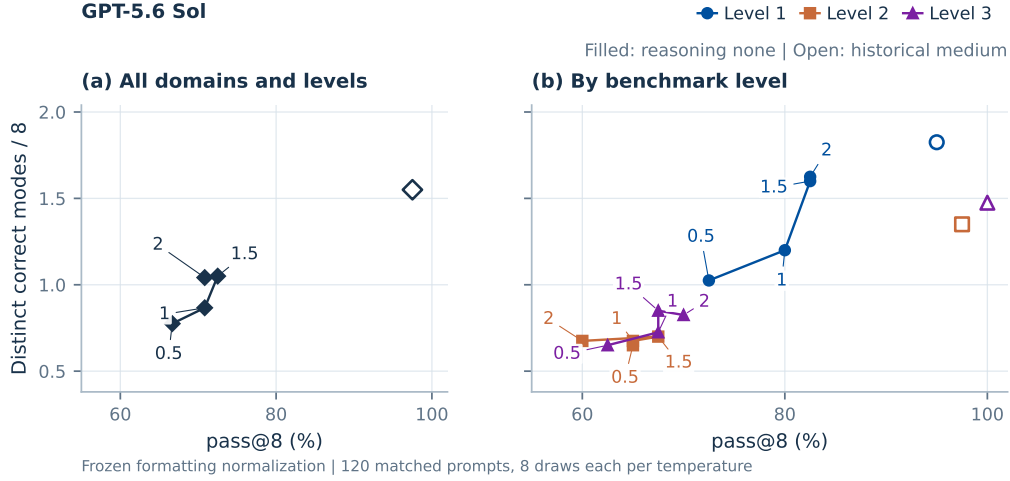


Figure 8: **Temperature traces pass@8 and verified breadth.** GPT-5.6 Sol, with 120 fixed prompts and eight draws each at four temperatures. Left: all domains and levels; right: separate levels, averaging five domains equally. Empirical pass@8 is the fraction with at least one correct draw; distinct@8 averages unique correct modes, including zeroes. Connected points use reasoning none and the frozen formatting normalizer. Open markers show the historical medium-reasoning reference on the same prompts (returned $T = 1$), unconnected because its configuration differs. This does not test temperature robustness with medium reasoning. Appendix H.3 gives pointwise uncertainty intervals for every level and all four measured temperatures.

6 RELATED WORK

RLHF and reasoning RL can reduce output diversity, entropy and high-budget coverage (Kirk et al., 2024; Dang et al., 2025; Cui et al., 2025; Yue et al., 2025). MODEBENCH measures verified output identities; semantic clustering instead infers equivalence from language (Farquhar et al., 2024). Sampling-based selection motivates preserving proposal alternatives (Brown et al., 2024; Snell et al., 2024; Cobbe et al., 2021; Lightman et al., 2024); our experiments do not measure their downstream utility.

Outcome bonuses, set prediction, distribution matching and uniform-correct objectives also encourage broader support (Song et al., 2025; Chen et al., 2025; Li et al., 2025; 2026a;b; Lochab et al., 2026). MaxRL combines objectives across sampling budgets (Tajwar et al., 2026); we ask whether memory adds value. Replay can mitigate forgetting (Lin, 1992; Rolnick et al., 2019), and RLEP reuses successful reasoning trajectories (Zhang et al., 2025). Our memory balances execution keys independently of discovery frequency, connecting replay to behavioral archives in quality-diversity methods (Mouret & Clune, 2015; Ecoffet et al., 2021).

7 CONCLUSION

MODEBENCH separates correctness from the availability of alternative verified outputs. Replay-MaxRL uses their execution identities as memory: fresh updates improve the policy on new samples, while replay keeps stored successes involved in training. Matched comparisons show that replay can improve correctness and sampled breadth, including beyond MaxRL.

Our claims concern sampled, task-defined outputs. Replay cannot retain a mode before discovering and storing it, and improved held-out breadth does not show that every stored exemplar survives. A fixed-bank audit finds improving central scores alongside a declining tail (Appendix F.4); exact mode probabilities remain unverified. The idealized theory does not certify neural training, and the experiments do not show that breadth causes the accuracy gains. Whether retaining alternatives helps agents recover or adapt remains an open question.